



TECHNICAL ASSESSMENT · AI ARCHITECT

PDF Name Extractor & RAG

From scanned PDFs to names, boxes and grounded answers:
what was built, how each choice was measured, and where it
goes on AWS.

30 → 0

failing provided tests

40

model configurations benchmarked

215

tests passing

166

AWS resources, previewed offline

Alvin Gregory

October 2026

01 • SPIKES

Measured before we chose

Each model was picked by a spike on a 150-page dataset with exact ground truth: the samples plus noise, skew, blur, JPEG and low-DPI scans.

OCR CHARACTER ERROR
14.9% → 2.4%

NER FALSE POSITIVES
178 → 0

RETRIEVAL RECALL@3
0.93 → 1.00

CACHE FALSE HITS
18 → 0



A OCR

150 pages × 3 reps · per page

ENGINE	CER	NAME RECALL	P99	COLD START
Tesseract (original)	14.9%	76.3%	416 ms	0.4 s
PaddleOCR native	1.5%	90.1%	3,800 ms	4.3 s
PaddleOCR · ONNX CPU	2.4%	87.9%	519 ms	0.8 s
PaddleOCR · ONNX CoreML	2.4%	87.9%	6,186 ms	10.3 s
Docling · MPS	35.7%	59.8%	3,033 ms	6.5 s

ONNX Runtime: near-native accuracy, 8× faster (p50), no GPU. Docling returns no word boxes.

B Person NER

on real OCR output · per page

MODEL	F1	FALSE POS.	P50	RSS
spaCy sm (original)	0.863	178	18 ms	0.4 GB
spaCy trf	0.933	33	76 ms	2.8 GB
BERT-NER · ONNX	0.913	53	36 ms	1.4 GB
GLiNER · torch	0.942	2	69 ms	1.9 GB
GLiNER · ONNX int8	0.940	0	32 ms	1.1 GB

int8 is 3.3× smaller (175 MB); threshold 0.3, recalibrated on this dataset, recovers fp32's F1.

C Embeddings

30 questions · 113 graded pairs

MODEL	DIM	RECALL@3	MRR	HARD-NEG AUC	QUERY
MiniLM (original)	384	0.93	0.89	0.39	1.2 ms
MPNet	768	0.93	0.85	0.57	4.5 ms
bge-small	384	1.00	0.92	0.58	2.4 ms
bge-base	768	0.97	0.89	0.63	4.5 ms
e5-base	768	0.97	0.92	0.55	4.1 ms

No model separates role swaps ("A reports to B" vs "B reports to A"), so the cache adds a guard.

D Answer cache

263 cases · 63 must-miss

RULE	REPEAT HITS	PARAPHRASE HITS	FALSE HITS
cosine ≥ 0.80	100%	90%	30
cosine ≥ 0.90	100%	48%	18
cosine ≥ 0.96	96%	16%	9
≥ 0.90 + QuestionGuard	94%	10%	0

The guard rejects differing negation, numbers, entities or entity order. A miss costs one LLM call; a false hit is a wrong answer.

Four technologies **carry the design**



ONNX Runtime
MODEL SERVING

All three models run on it: **PaddleOCR**, **GLiNER int8** and **bge-small**. CPU only, Fargate-friendly, small images, no PyTorch or Paddle at runtime for OCR and embeddings.

8×
faster than native Paddle



Temporal
DURABLE WORKFLOWS

Async jobs: pages OCR'd **in parallel** on a **cpu** queue, DB and vector writes on an **io** queue, retries with backoff, heartbeats, and a reconciler for jobs that never started.

3.5 s
2-page job, end to end



Qdrant
VECTOR STORE

Document chunks and the **semantic answer cache**. Content-addressed IDs index a PDF once; cache entries are scoped to the corpus version, so new documents invalidate stale answers.

94%
repeat questions served from cache



Pulumi
INFRASTRUCTURE • PYTHON

The AWS target as **components** with dev and staging configs. Validated **offline**: unit tests with mocks, a CrossGuard policy pack, and previews without AWS credentials.

166
resources in the dev stack

03 · DESIGN PATTERNS

Patterns that kept it swappable and fast

Every backend sits behind a Protocol and is injected; models load once per process.

FACTORY

Engines by config

Registries build the configured OCR, NER and embedding engine. **chosen** and **fallback** stacks install as uv extras.

```
OCR_ENGINE=rapidocr # | tesseract
ocr = OCRServiceFactory.create(settings)
```

CONNECTION POOLING

No I/O pile-ups

Bounded Postgres pools only in **api** and the **io worker**; CPU workers hold none, so they scale without adding connections. A busy pool waits at most 2 s, then answers **503 + Retry-After** instead of hanging.

```
AsyncConnectionPool(max_size=5, timeout=2,
max_waiting=50, check=check_connection)
```

CACHE

Pay once

Semantic answer cache with a question guard. Refusals never cached. OCR **once** per request. PDFs indexed once by **content hash**.

```
hit = cosine ≥ 0.90 and guard.same(q, q')
doc_id = uuid5(sha256(pdf))
```

DRY

Written once

One **DocumentIndexer** for /api/ingest and the workflows; one **ExtractionSession** for sync and async; shared upload and SSE helpers.

```
indexer.embed(text) # cpu worker
indexer.store(id, chunks) # io worker
```

TRANSACTIONAL OUTBOX

PLANNED

No lost jobs

Job row and outbox row in **one transaction**; Debezium → Kinesis → Lambda starts the workflow. Today: direct start + reconciler.

```
INSERT jobs, outbox # one tx
→ Debezium → Kinesis → Lambda → Temporal
```

Same containers, **managed** around them

— request / data - - - planned (outbox)

